The next URL is more complex: its authority component contains both a "userinfo" subcomponent (which is a username and password combination delimited from the rest of the component by the `@` character) and a TCP port number (delimited by the `:` character). An HTTP client using this URL is going to connect to TCP 4430 (not the standard HTTPS port of TCP 443). There is also the query component (delimited by `?`) which would assign a value of `"How_the_internet_works"` to a `"name"` parameter (which is exposed via a superglobal). What about that userinfo subcomponent of `pault:notsosecret`? Well, there was a time when that data would end up in an HTTP header and be used for basic authorization, but browser support has long since been withdrawn (and rightly so). Userinfo data is not made available anywhere in PHP.

Finally, purely for illustration and comparison, the URL `file:///Users/...` uses the "file" scheme and has no authority component at all, which is fine. A URL component without characters is considered to be empty, and the only component that cannot be empty is the scheme. So, in this case, the local system is assumed to be the recipient of the request, the path is `/Users/pault/Documents/local_file.php`, and a URL like this, if used with a web browser, would simply have it open the file. All that's going to happen here though is the contents of `local_file.php` will be rendered as text (web browsers don't execute PHP, nor should they).

Those of you vaguely familiar with how URLs work might be wondering about query strings at this point. Query strings are another data-passing method that operate via the path portion of the URL string and are appended with a `?` separator character – `https://some.domain.com/index.html?foo=bar` is a URL with a query string `foo=bar`. Query strings are not part of the HTTP specification as such: provision was made for them in URL syntax in the original RFC, but they were otherwise dealt with in a separate document (RFC 3986). Even then no formal standard defines their operation (although the standards do assert what they *shouldn't* do), and they operate purely on well-established conventions. We'll get to those in a moment when we look at resource paths in more detail; meanwhile, let's examine what happens when an HTTP request is made.

HTTP(S) Requests

From the perspective of PHP, the main components of an HTTP(S) request message are *method*, *resource path*, *headers*, and *body*. In addition, a special header-based mechanism exists (as an extension to the HTTP specification) known as *cookies* (you've probably heard of them). The client and server communicate using the HTTP protocol,